CNVS, Technical Document

Construction of the Measurement Grid, Recursive Decomposition, and Terminal Selectors in CNVS

1. Purpose of the document

This document describes the operational functioning of the measurement grid within the CNVS framework. Its purpose is to clarify how original data are transformed into a measurable structure, recursively fragmented, and assigned to local verifiers without allowing them to access the global meaning of the data, the complete state structure, or the hidden binding of the invariants.

In CNVS, distributed validation does not occur through consensus, voting, quorum, threshold mechanisms, or external reconstruction. Local verifiers operate only on terminal fragments (leaves), while systemic reconstruction and state acceptance occur internally through the global validation function, relational consistency, hidden invariant binding, and the Global Veto.

2. Fundamental principle

The original data are not treated as semantic text, but as a decomposable typed structure.

Complex data, such as a clinical record, a financial file, an asset registry, or a laboratory record, are not submitted to verifiers in the form:

> **“Check the glucose value of patient X.”**

Instead, the data are transformed into an internal structural representation, similar to a typed tree or typed graph, in which each item of information is placed along a structural path.

Abstract example:

```
Candidate state
└── Folder_6674
    └── Exam_56378
        └── Analyte_15
            └── Value
                ├── Integer_part
                └── Decimal_part
```

The local verifier does not know the semantic names of these fields. In pure CNVS, the semantic path is replaced by an opaque selector.

Non-opaque example:

```
Folder[6674].Exam[56378].Analyte[15].Value.Decimal
```

Opaque CNVS example:

```
τ_9F31A7.C_2.t
```

Only the internal system knows the real meaning of that selector and the reconstruction tree.

3. Technical vocabulary

3.1 Original data

Original data are the initial information inserted into the system. They may be a document, a record, a table, a clinical file, an asset declaration, a database, or any informational structure submitted for validation.

In CNVS, the original data are not directly validated as a single block. They are transformed into a candidate state and then decomposed.

3.2 Candidate state

The candidate state is the structured representation of the original data within the system.

It does not simply coincide with the raw data. It is a formalized configuration containing:

- structural elements;
- internal relations;
- types;
- expected local observations;
- declared properties;
- consistency constraints;
- internal invariants.

The candidate state can be accepted only if it passes global validation.

3.3 Typed tree or graph

The data are converted into a typed tree or typed graph. Each node represents a portion of the structure, while each leaf represents a potentially measurable terminal element.

Example:

```
S
├── s_1
│   ├── s_1.1
│   └── s_1.2
└── s_2
    ├── s_2.1
    └── s_2.2
```

This structure need not be human-readable and, in security-sensitive implementations, should not expose semantic paths to local verifiers. It may be encoded through internal identifiers, opaque coordinates, or automatically generated selectors.

3.4 Recursive decomposition

Recursive decomposition is the process by which the candidate state is divided into increasingly smaller fragments.

Formally, it corresponds to the decomposition map:

$$\mathfrak{D}(s)$$

which associates a structural element with its sub-elements.

Decomposition continues until terminal fragments are reached, i.e. elements that are not further decomposed for that specific validation instance.

3.5 Terminal fragments

Terminal fragments are the leaves of the decomposition tree. They are the minimal units assigned to local verifiers.

A terminal fragment may be:

- a field;
- a sub-field;
- a digit;

- an integer part;
- a decimal part;
- a partial string;
- a bit;
- a structural component;
- a local measurement;
- a hash commitment;
- an atomic property.

The essential point is that a terminal fragment, by itself, must not possess sufficient systemic meaning.

3.6 Measurement grid

The measurement grid is the operational vocabulary by which the system defines what must be measured locally.

It is not a list of natural-language questions. It is a set of automatically generated structural directives.

Human example:

```
Take folder number 6674 and report the decimal part of the value of analyte 15.
```

CNVS example:

```
Execute V_L on selector τ_i at time t.
```

The verifier receives only the second form, not the complete semantic description.

3.7 Terminal selector

The terminal selector is the opaque address that identifies a terminal fragment in the internal structure.

Example:

```
τ_i
```

or:

```
τ_9F31A7
```

The terminal selector replaces the semantic path readable by the verifier, so that the external verifier acts only as an executor of a local measurement procedure over an assigned selector.

Real internal path:

```
Folder[6674].Exam[56378].Analyte[15].Value.Decimal
```

External selector assigned to the verifier:

```
τ_9F31A7
```

The verifier does not know the correspondence between the selector and the actual full path. It can execute the assigned task only through the local resolver or measurement interface exposed by the system, without accessing the semantic path behind the selector.

3.8 Local verifier

The local verifier is the subject, node, machine, or module that performs verification on a terminal fragment.

In CNVS, the local verifier:

- does not vote;
- does not participate in consensus;
- does not reconstruct the global state;
- does not know the global function V_G;
- does not know the hidden binding of the invariants;

- does not necessarily know the semantic meaning of the fragment (for example, it could be tasked with reporting the last 10 characters of a PDF document to be validated, at the specified coordinates, without granting access to the entire document or page);
- returns only local evidence.

3.9 Local evidence

Local evidence is the result produced by the verifier after executing V_L.

It may include:

- local outcome;
- observation;
- hash commitment;
- signature;
- timestamp;
- proof of execution;
- limited metadata;
- local measurement.

Local evidence is necessary but not sufficient for global state acceptance.

3.10 Hidden invariant binding

Hidden invariant binding is the internal structure that connects fragments, coordinates, properties, relational dependencies, and global validity conditions. It is represented internally through the relation set R(t), the invariant family C, and the hidden binding map that associates terminal evidence with the relevant global constraints.

Local verifiers do not know of these connections.

This means that a verifier can correctly measure a fragment without knowing:

- which other fragment is connected to it;
- which global constraint involves it;
- which part of the state it contributes to validating;
- which systemic error it may generate;
- which combination of fragments is critical.

3.11 Global validation function

The global validation function, denoted as:

```
V_G
```

determines whether the candidate state can be accepted.

It is not the sum of the local results. It is not a vote. It is not a quorum. It is not consensus.

V_G evaluates:

- local admissibility of terminal fragments;
- relational consistency;
- satisfaction of invariants;
- hidden binding;
- absence of systemic violations;
- Global Veto.

3.12 Global Veto

The Global Veto prevents a set of positive local verifications from being sufficient to accept the state.

Even if all local verifiers return apparently valid outcomes, the state may be rejected if it violates a global condition, an internal relation, or a hidden invariant.

This is one of the central points of CNVS:

```
local validity ≠ global validity
```

4. CNVS operational pipeline

Phase 1 — Insertion of the original data

The process begins when data are inserted or declared in the system.

Examples:

- clinical record;
- laboratory test;
- asset declaration;
- custody registry;
- transaction;
- state of an industrial plant;
- financial record;
- administrative document.

The inserted data are not immediately accepted as valid. They become a candidate state.

Phase 2 — Normalization and canonicalization

The original data are converted into a canonical form.

This phase prevents ambiguities due to:

- different formats;
- different orderings;
- equivalent representations;
- separators;
- units;
- missing fields;
- encoding;
- numerical precision;
- date/time formats.

Example:

```
114,85
```

may be normalized as:

```
114.85
```

or as the structure:

```
{
 "integer_part": "114",
 "decimal_part": "85"
}
```

This phase is essential because the subsequent decomposition must operate on a stable structure.

Phase 3 — Construction of the typed structure

The normalized data are transformed into a typed tree or typed graph.

Example:

```
Record
└── Exam
    └── Analyte
        └── Value
            ├── Integer_part
            └── Decimal_part
```

Each node has a type. For example:

```
Record
Exam
Analyte
NumericValue
IntegerComponent
DecimalComponent
```

The system does not treat the data as a sentence, but as a structure.

It is the sole responsibility of the entity compiling and attaching the data to the archive for the first time (e.g., a hospital or financial institution) to prepare the structural coordinates needed to build the selection tree. During the acquisition phase, the original documents must undergo complete semantic masking (source tokenization). Each keyword, header, or sensitive data (e.g., patient name, test type, analyte name) is replaced and obscured with a unique alphanumeric identifier (e.g., ID_991, TEST_404). This initial mapping is a fundamental prerequisite for high-opacity CNVS deployments and allows the algorithm to generate a measurement grid without exposing explicit semantic labels to local verifiers. The document entering the verification circuit will therefore be devoid of any obvious human semantics, supporting privacy by design.

Phase 4 — Recursive decomposition

The system applies the decomposition map $\mathfrak{D}$.

A field may be decomposed into sub-fields.

Example:

```
Value = 114.85
```

may become:

```
Value
├── Integer_part = 114
└── Decimal_part = 85
```

The integer part may be further decomposed:

```
114
├── 1
├── 1
└── 4
```

The decimal part may be decomposed:

```
85
├── 8
```

```
└── 5
```

The depth of decomposition depends on the required level of epistemic isolation.

Phase 5 — Generation of terminal fragments

When decomposition reaches the established level, the system identifies the terminal leaves.

Example:

```
f_1 = Integer_part
f_2 = Decimal_part
```

or, at a deeper level:

```
f_1 = digit 1
f_2 = digit 1
f_3 = digit 4
f_4 = digit 8
f_5 = digit 5
```

These terminal fragments are the units that can be assigned to distinct local verifiers.

Phase 6 — Generation of opaque selectors

Each terminal fragment is associated with an opaque selector.

Internal path:

```
Folder[6674].Exam[56378].Analyte[15].Value.Decimal
```

Generated selector:

```
τ_9F31A7
```

The selector is what is sent to the verifier.

The verifier does not receive the complete semantic path. It receives only an operational directive.

Phase 7 — Construction of the measurement grid

The measurement grid is the set of generated terminal tasks.

Example:

```
G_t = {
 (τ_1, V_L, t),
 (τ_2, V_L, t),
 (τ_3, V_L, t),
 ...
}
```

Each element of the grid specifies:

- which selector to measure;
- which local function to apply;
- at which time t;
- which form of evidence to return.

The measurement grid is generated automatically from the data structure and the decomposition map.
The measurement grid transforms the verifier into a mere physical meter, procedurally guided by the algorithm. The instructions do not require context interpretation, but pure mechanical execution on a pre-masked matrix. For example, the algorithm issues the following directive: 'Go to page 3, column 4 of the document, locate the opaque identifier ID_884, and report only the second and third digits of the associated numeric value' (e.g., the decimal

part). The verifier operates exclusively on spatial coordinates and alphanumeric codes, without ever exposing itself to the clinical or financial significance of the data it is manipulating.

Phase 8 — Randomized assignment to local verifiers

Terminal tasks are assigned to local verifiers.

The assignment must be:

- unpredictable;
- not semantically informative;
- not necessarily uniform;
- compatible with the security conditions of the model.

Example:

```
τ_1 → verifier A
τ_2 → verifier B
τ_3 → verifier C
```

Verifier A does not know that τ_1 may be the integer part of a value. Verifier B does not know that τ_2 may be the decimal part of the same value.

When full selector opacity cannot be achieved in a given implementation, the weaker requirement is that the verifier's local view must still remain insufficient to reconstruct the complete semantic meaning of the observed information, including its specific analyte, test, patient, or systemic role.

Phase 9 — Execution of local verification

Each local verifier executes V_L on the assigned fragment.

Abstract form:

```
V_L(f_i) = (b_i, P_i)
```

where:

```
b_i = local outcome;
P_i = set of observed local properties.
```

In an implementation, the verifier may return:

- local measurement;
- commitment;
- salted hash;
- signature;
- proof of access;
- timestamp;
- Boolean outcome;
- limited metadata.

It is important not to use a naked hash of small-domain values. For example, values such as 85, M, 0, 1, or October are vulnerable to dictionary attacks.

It is preferable to use commitments of the form:

```
H(value || salt_i || τ_i || t)
```

or an equivalent scheme in which the salt or binding is not accessible to the attacker.

If this is not possible, it is recommended to apply the principle of semantic blindness by exposing the observed object without acquiring any material that could aid in reconstructing the raw data (additional documentation, technical specifications, accessory modules, wiring diagrams, storage environment, etc.).

Phase 10 — Collection of local evidence

Local evidence is collected by internal nodes or the validation module and is not under the control of a human authority, but belongs to the application domain of the V_G function applied to the partial states.

Therefore, this collection does not constitute consensus.

Intermediate nodes do not decide global validity by voting. They may:

- order evidence;
- verify signatures;
- check message integrity;
- aggregate commitments;
- forward proofs;
- eliminate malformed responses;
- prepare inputs for V_G.

The global decision remains internal to CNVS.

Phase 11 — Internal systemic recomposition

Recomposition does not mean that a verifier reconstructs the complete data.

Recomposition occurs internally through:

- selector map;
- hidden binding;
- relations among fragments;
- invariants;
- topological consistency;
- global function V_G.

Example:

```
τ_1 internally corresponds to Integer_part
τ_2 internally corresponds to Decimal_part
```

The system knows that:

```
τ_1 and τ_2 belong to the same numeric value
```

but local verifiers may not know this depending on the degree of epistemic isolation achieved.

Phase 12 — Global validation

The function V_G evaluates whether the candidate state is acceptable.

A state is accepted only if:

- the required terminal fragments are locally admissible;
- the internal relations are coherent;
- the hidden invariants are satisfied;
- the binding among fragments is valid;
- the Global Veto is not activated.

If a global condition fails, the state is rejected even if many local verifications are positive.

5. Operational example

Original data:

```
Analyte 15 value = 114.85
```

Internal representation

```
Analyte_15
└── Value
    ├── Integer_part = 114
    └── Decimal_part = 85
```

Decomposition

```
f_1 = 114
f_2 = 85
```

Opaque selectors

```
f_1 → τ_A71F
f_2 → τ_B92K
```

Assignment

```
τ_A71F → Verifier 1
τ_B92K → Verifier 2
```

Received directives

Verifier 1:

```
Execute V_L on τ_A71F at time t.
```

Verifier 2:

```
Execute V_L on τ_B92K at time t.
```

Neither verifier receives:

```
Analyte_15
Value = 114.85
Integer part
Decimal part
Clinical record
```

Local responses

```
Verifier 1 → local evidence E_1
Verifier 2 → local evidence E_2
```

Internal validation

The internal system knows the binding:

```
τ_A71F ↔ Integer_part
τ_B92K ↔ Decimal_part
```

and evaluates whether:

```
Integer_part || Decimal_part
```

satisfies the required invariants.

Final validity is not decided by the two verifiers, but by V_G.

6. Difference from consensus and threshold systems

In classical consensus:

```
validity arises from agreement among nodes.
```

In threshold systems:

```
reconstruction arises from possession of enough shares.
```

In CNVS:

- terminal nodes do not decide;
- terminal fragments do not reconstruct by themselves;
- global validity is not reducible to local responses;
- systemic reconstruction remains internal;
- hidden invariants determine state acceptance.

This is the fundamental difference.

7. Security property obtained

CNVS security derives from the combination of:

- recursive fragmentation;
- epistemic isolation;
- opaque selectors;
- randomized assignment;
- hidden binding;
- bounded topological leakage;
- residual min-entropy of missing fragments;
- Global Veto.

An attacker does not merely need to compromise a node.

The attacker must reconstruct:

- which selectors correspond to which fragments;
- which fragments are critical;
- which fragments are correlated;
- which internal binding connects them;
- which invariants must be satisfied;
- which combination produces global validity.

The difficulty of the attack does not depend only on the number of compromised nodes, but on the amount of global information that remains non-inferable.

Even if colluding attackers were able to gather enough information to infer that the value 114.85 refers to a cholesterol-related analyte, this information would remain insufficient by itself unless it could also be correlated with the hidden invariants, the relevant patient-specific context, and the expected relational coherence.

8. Essential implementation conditions

For this architecture to remain coherent with CNVS theory, certain conditions are indispensable.

8.1 Opaque coordinates

Verifiers should receive semantically unreadable paths.

To avoid:

```
Folder[6674].Exam[56378].Analyte[15].Value.Decimal
```

To prefer:

```
τ_i
```

or:

```
selector_id = τ_i
```

8.2 Non-trivial commitments

Simple hashes over small domains must not be used.

To avoid:

```
H(85)
```

To prefer:

```
H(value || salt_i || selector_i || t)
```

or equivalent commitments.

8.3 Non-exposed internal binding

The mapping between opaque selectors and their full meaning must remain internal.

The attacker must not know the mapping, and epistemic isolation must be maximized.

8.4 Non-revealing topology

Task distribution must not easily reveal which fragments are correlated.

For example, if two linked fragments are always sent to nearby nodes or in the same order, the topology itself may become leakage.

8.5 Protected V_G

The global validation function must not become a trivial point of compromise.

It must be designed so that:

- the binding remains hidden;
- the invariants are not publicly reconstructible;
- local evidence is verifiable;
- the Global Veto cannot be bypassed.

9. Final conceptual formula

CNVS can be summarized as follows:

```
Original data
→ normalization
→ candidate state
→ typed tree/graph
→ recursive decomposition
→ terminal fragments
→ opaque selectors
→ measurement grid
→ randomized assignment
→ local verification V_L
→ local evidence
→ internal recomposition
→ progressive internal V_G checks
```

→ global V_G over the candidate state

→ state acceptance or veto

In shorter form:

The verifier measures a leaf.

The system interprets the forest.

10. Conclusion

The measurement grid in CNVS is not a set of human questions, but a structural vocabulary automatically generated from the decomposition of the original data.

The data are transformed into a typed tree or graph, recursively divided into terminal fragments, converted into opaque selectors, and assigned to local verifiers. The verifiers produce local evidence without knowing the global meaning of the fragment, the hidden binding of the invariants, the relations, and the global validation function.

Systemic reconstruction does not occur inside the verifiers. It occurs internally within CNVS through relational consistency, hidden binding, invariants, and the Global Veto.

In this way, CNVS radically separates:

local measurement

from

global state validity

and defines a form of distributed validation in which systemic acceptance is not produced by consensus, quorum, threshold, voting, or joint computation, but by an internal validation function over epistemically isolated fragments.

Note on observing physical objects: The semantic masking (tokenization) mechanism described in this document is essential for data flows and written documents, where information is natively explicit. However, the principle of system blindness can also apply to the direct inspection of physical objects or complex infrastructures, provided that the verifier lacks the internal schema, functional labels, relational topology, invariant parameters, and hidden validation binding. or complex infrastructures (e.g., a nuclear reactor, an industrial plant). In the physical domain, the deliberate absence of technical specifications, wiring diagrams, operating manuals, or explanatory labels on components can render the observed feature insufficient for reliable semantic reconstruction by the observer. The verifier tasked with measuring an isolated parameter (e.g., pressure on an analog dial) lacks sufficient relational information to translate that visual observation into a global semantic or systemic meaning, ensuring epistemic isolation directly at the source.

Physical observability is therefore not equivalent to semantic reconstructability.

USE CASE

Exploratory Use Case: CNVS-Based Fragment Validation for Network Transport

This section is exploratory and does not claim a drop-in replacement for existing transport protocols. It illustrates how the CNVS principles of recursive decomposition, local verification, hidden invariant binding, and Global Veto may be interpreted in a high-assurance transport-validation setting.

1. The Temporal Paradigm: The Evolution of Epsilon (ε_a)

In the transition from formal theory to real-world network engineering, the concept of delay (measured in milliseconds, latency, and jitter) requires an operational translation of the temporal constraint.

- **In the Paper (Asymptotic Model):** ε_a is defined as a static, theoretical limit. It represents the formal safety asymptote, the impassable boundary beyond which the system considers the spatiotemporal entropy to be compromised.
- **In Telematics Routing (Operational Model):** ε_a becomes dynamic ($\varepsilon_{dynamic}$). The network card calculates this value in real time based on the standard deviation (variance) of the specific topological route.
- **The Golden Rule:** Engineering ensures that dynamic tolerance never exceeds the theoretical asymptote ($\varepsilon_{dynamic} \leq \varepsilon_a$). The system fluctuates to absorb real-world friction, but without exceeding the theoretical safety bound assumed by the model.

2. Congestion Management: Fractal Reshape and Granularity Limit

Unlike classical transport architectures, a CNVS-inspired validation layer does not accept reconstructed candidate states containing unresolved required terminal losses. If congestion causes εdynamic to exceed, a topological mutation is triggered:

- **Invalidation and Regeneration:** The blocked or delayed terminal fragments (the "leaves" in the $S^{(n)}$ layer) are regenerated and share the same invalidated parent node. The system ascends the tree to the parent node in $S^{(n-1)}$.
- **Fractal Fragmentation:** The parent node regenerates the payload, forcing secondary fragmentation. The new fragments become lighter and smaller, channeling themselves on alternative routes to bypass the blockage.
- **Granularity Limit:** This iterative process of "streamlining" packets to overcome latency continues until the maximum granularity limit is reached (the minimum fragment size that still guarantees the C Invariant and the preservation of the information necessary for local verification).

3. Verification Architecture: Local and Global Level

The packet validation and reconstruction process follows a strict hierarchy to prevent attacker inference.

• **Hardware Local Verification (V_L)**: There is a strict 1-to-1 ratio. A single fragment undergoes a single local validation. This validation is managed at the hardware level with random (stochastic) assignment distributed across multiple independent channels, reducing the attacker's ability to predict or correlate the route.
• **Global Verification (V_G)**: This occurs through backward reconstruction. The system assembles the partial states (starting from the validated leaves in $S^{(n)}$) and ascends the tree until the topology closes at the original level, verifying global and partial semantic consistency during reconstruction.
• **Parameter Transfer and Veto:** During reconstruction, the internal state parameters are transferred. The check focuses on the internally hidden and never revealed θ_C parameters. If the θ_C parameters are violated (e.g., missing or tampered fragments), the system does not attempt a partial recovery: it immediately triggers the Global Veto, declaring the system in the Rejected State. Reconstruction is interrupted to protect the Invariant.

4.1 Local Verification (V_L) and Dynamic Tolerance (ε_a)

Local Verification (V_L) operates at the level of the individual terminal fragment $s \in Ter(t)$. Its sole purpose is not semantic data analysis, but rather rigorous control of the kinematic convergence of the packet. Specifically, V_L calculates and validates the transmission time (measured in milliseconds) as a nonlinear function of two physical variables: the topology of the network path assigned to the fragment and the volume (payload) of the fragment itself.
In the ideal asymptotic model, the safe time limit is defined by a static constant. However, when implementing on asynchronous networks subject to congestion and jitter, delay tolerance must be parameterized. Recalling the definition of quantitative convergence on metric spaces (Formula 8):
$Conv_\sigma(a, d, obs) = 1 \Leftrightarrow \boldsymbol{\delta}_\sigma(d, obs) \leq \varepsilon_{dynamic}$.
Where the function $\boldsymbol{\delta}_\sigma(d, obs)$ measures the actual time gap (the recorded latency) between the observation and the expected data. To prevent the system from collapsing into false positives due to natural network friction, the parameter εa evolves from a theoretical asymptote to a dynamic tolerance threshold ($\varepsilon_{dynamic}$). This value is calculated statistically based on the standard deviation of the specific topological route traversed by the fragment. Local Verification is successful only if the fragment's delay falls within this fluid tolerance window. Consequently, the local admissibility of a state fragment s is formalized as:
$Adm_L(s) = 1 \Leftrightarrow \pi_1(V_L(s)) = 1$
Exceeding the εdynamic threshold leads to a lack of spatiotemporal convergence and the consequent invalidation and regeneration of the delayed or blocked fragment, forcing a topological reshape on the parent node to circumvent the congestion, and subsequently reshaping it on another transmission channel.

4.2 Global Verification (V_G) and Protection of the Static Invariant C.

While V_L handles local spatiotemporal convergence constraints, V_G operates in the logical-mathematical domain of global reconstruction and invariant satisfaction. Its role is not to evaluate latencies, but to perform a backward reconstruction of the topological tree, assembling the partial states (validated terminal fragments) until the original information payload is restored.
V_G acts as a deterministic veto layer for candidate-state acceptance. It constrains acceptance of the final state to the simultaneous satisfaction of three exact conditions (Axiom III):
$V_G(S(t)) = 1 \Leftrightarrow (\forall s \in Ter(t), \pi_1(V_L(s)) = 1) \wedge Cons_R(R(t), E(t)) \wedge Inv_C(S(t))$
The Global Verification process is therefore structured on the following sequential checks:
1. **Topological Closure:** All terminal fragments Ter(t) generated by the reshape must have successfully passed Local

Verification ($\forall s \in Ter(t), \pi_1(V_L(s)) = 1$). The loss of a required terminal fragment prevents acceptance of the corresponding reconstructed candidate state unless the system successfully performs reshaping, reassignment, or regeneration before global evaluation.
2. **Relational Consistency:** The internal structural relations and entropy of the system ($Cons_R(R(t), E(t))$) must remain consistent with each other during the transfer and with the partial state parameters (which are represented by $c_i \in C$).
3. **Invariant Matching:** This is the last and most critical level of control. The reconstructed system S(t) must exactly and deterministically satisfy the Global Static Invariant (Inv_C). Invariant C is absolute, devoid of temporal coordinates.
If, during reconstruction, the critical parameter θ_C is violated due to semantic or structural alterations, the V_G triggers a Global Veto. The system is instantly downgraded to the Rejected State, aborting the operation to protect Invariant C from any inference attempts, including attacks against the network's temporal metrics.

Implementation Note: Algorithmic Execution of CNVS as a Transport-Validation Layer

From a strictly axiomatic point of view (Axiom III), the Global Invariant C is satisfied if and only if all sub-invariants $c_i \in C$ hold true over the entire state S(t). However, in the practical application of the CNVS framework on asynchronous networks (in clear discontinuity with the end-to-end approach typical of TCP/IP protocols), the resolution of this logical conjunction does not occur simultaneously at the end of transmission.
To improve computational efficiency and reduce unnecessary propagation of invalid partial states, a CNVS-inspired transport-validation implementation may realize Global Verification (VG) as a bottom-up iterative algorithm.
The execution mechanics follows the following operational steps:
1. **Partial Degree Validation:** Payload reconstruction does not wait for the reception of all terminal fragments at layer S(0). Instead, it proceeds by partial assembly. The terminal fragments of the $S^{(n)}$ layer that pass Local Verification (V_L) converge at the respective parent node at the $S^{(n-1)}$ layer.
2. **Early Verification of c_i:** Each intermediate parent node, as soon as it reconstructs its partial state, computes only the subset of ci invariants within its topological or semantic scope.
3. **Fail-Fast and Immediate Global Veto:** In the standard TCP/IP protocol, a corrupted packet (or a failed checksum) is often detected only at the destination, wasting precious bandwidth for transporting already compromised data. In CNVS, if an intermediate node detects the violation of even a single ci sub-invariant in the partial state, the system does not proceed with the reconstruction of the higher layers. The Global Veto is triggered instantly at the local level.

This implementation architecture transforms the atemporal logical constraint of Axiom III into an active network filter. It prevents the propagation of invalid, corrupted, or attacker-manipulated information states to higher nodes in the hierarchy, potentially reducing the consumption of computing resources and ensuring that only states satisfying the required invariant checks reach the $S^{(0)}$ level.

5. Structural Differences: CNVS vs. TCP/IP

The application of CNVS principles to telematics routing departs from the sequential and end-to-end assumptions of classical transport architectures.

Characteristic	TCP/IP Protocol	CNVS Architecture (Telematic Domain)
Management Latency / Delays	Waits for a timeout, declares the packet lost, and requests retransmission (vulnerable to latency attacks).	Uses a $\varepsilon_{dynamic}$. If unsuccessful, it invalidates the affected terminal fragments and triggers reshaping, reassignment, or regeneration through alternative channels.
Payload Integrity	Uses Forward Error Correction (FEC) and expendable packets.	Zero tolerance. Invariant C is an exact geometry: all fragments must arrive, otherwise the Global Veto is triggered. **The Global Veto applies to the acceptance of a reconstructed candidate state, not necessarily to every transient transport failure. Packet loss may trigger reshaping, reassignment, or regeneration before the candidate state is evaluated globally.**
Channel Security	Packets travel as whole packets (or in large segments) on standard BGP routes, exposing traffic metadata and, in insecure deployments, payload contents to interception or traffic analysis.	The fragments are randomly assigned 1 to 1 on separate hardware channels. The attacker's correlation and reconstruction capability is reduced by assignment entropy, route diversity, and bounded metadata leakage.
Verification Logic	The checksum only verifies binary integrity at the end of transmission.	Bifurcated: $V_L$hardware for spatiotemporal convergence of the single fragment, and V_G for semantic reconstruction backwards.

Architectural Synthesis: CNVS as a Transport-Validation Architecture

A telematic-engineering implementation of the CNVS framework may reduce reliance on conventional end-to-end transport assumptions by introducing a validation layer based on non-uniform fragmentation, opaque routing, and bottom-up invariant checks.

This transition is founded upon three operational pillars:

1. Non-Uniform Fractal Fragmentation (Overcoming Sequentiality)

The TCP/IP protocol divides payloads into uniform segments (e.g., 1460 bytes) and numbers them in cleartext, exposing the network to *Session Hijacking* and *Traffic Analysis* attacks. The CNVS, conversely, dismantles the data at the source (Ingress node) by applying a non-uniform randomized fragmentation:

- Terminal fragments at the $S^{(n)}$ level do not possess fixed dimensions. Blocks of variable weight (e.g., 20 bytes, 150 bytes, 1460 bytes) are generated up to the maximum limit allowed by the physical network's MTU, preventing secondary fragmentation on intermediate routers.
- The reconstruction order is not dictated by a cleartext sequence number, but is protected within the hidden topological binding encoded by R.
- Fragments are routed stochastically (randomly) over independent physical bands or logical channels. To a listening attacker (*Man-in-the-Middle*), the flow becomes harder to correlate, reconstruct, or interpret from observable traffic alone.

2. Binary Semantics: Static Invariants (c_i)

To bypass the computational overhead associated with high-level human semantic analysis (e.g., recognizing an image pattern), the CNVS applies the sub-invariants c_i directly to the Boolean algebra of the packets. The Invariant C operates as a set of hardware cryptographic rules, including:

- **Volumetric Invariant (Byte Summation):** A mathematical rule linking separated nodes. For example, the system verifies that the exact sum of the weight of fragment x and fragment x+3 (which may have arrived via two different channels) equals precisely 2827 bytes. The alteration or dropping of a single packet collapses the equation.
- **Inter-Node Logical Distance:** *Bitwise* operations verifying the alignment between adjacent fragments (e.g., checking the specific state of bit number 220 of fragment x in strict relation to bit 129 of fragment x+1).
- **Pattern Anchor (Target Sequences):** The search for specific binary strings (e.g., 001100011000) hidden at random offsets within the packets, acting as stochastic integrity micro-signatures.

3. Backward Reconstruction in Partial States (Bottom-Up)

A potential efficiency advantage over classical end-to-end validation arises from bottom-up partial-state verification.

- The CNVS does not wait for the reception of the entire payload to verify a final checksum. The algorithm proceeds by partial assemblies: the terminal fragments converge into their respective intermediate parent nodes.
- At each recomposition cycle (e.g., transitioning from $S^{(n)}$ to $S^{(n-1)}$), the intermediate node exclusively calculates the binary rules (byte summations, target patterns) applicable to that specific partial state.
- If an attacker alters a single bit during transport, the test fails locally. The system does not attempt corrections and aborts the calculation of higher levels, instantaneously triggering the Global Veto. The candidate reconstruction is aborted before higher-level acceptance of the manipulated state.

Engineering Conclusion

The CNVS interpretation of network transport should be understood as an experimental high-assurance validation layer rather than as an immediate replacement for existing transport protocols. By applying recursive non-uniform fragmentation, local verification, opaque selector assignment, and bottom-up invariant checks, the model suggests a possible architecture for transport-level integrity validation under explicit epistemic and topological constraints.

Its practical feasibility depends on implementation-specific factors, including routing diversity, fragment scheduling, congestion behavior, selector opacity, metadata leakage, commitment design, and the computational cost of progressive V_G evaluation.

AI Use Statement

AI-assisted tools were used during the preparation of this document for language editing, formatting, consistency checking, and technical revision support.
The ideas, theoretical architecture, formulas, definitions, conceptual structure, implementation logic, and scientific claims presented in this document are the original work of the author.
AI tools were used only as auxiliary instruments for review and presentation.
The author reviewed and approved the final document and assumes full responsibility for its content, correctness, originality, and interpretation.

Author: Massimo Comitato
Italy, Milano (MI),
04-07-2026